General - RPS Data

NIIT TRAINING DIVISION | LTM - iRUN Full Stack Engineering

TRAINING LAB GUIDE

LAB EXERCISE - 3

Brokers, Replication & Delivery Guarantees — Survive a Broker Failure

Read Leaders and ISR · Stop a Broker · Test acks and min.insync.replicas · Recover and Rebalance Leaders

Module Series	Apache Kafka Operations & Streaming Platforms — Day 4
Lab Type	Guided Hands-On SOP — Kafka Replication & Failure Handling
Duration	48 minutes (5 min setup + 25 min tasks + 10 min validation + 8 min debrief)
Difficulty	⭐⭐⭐ `Beginner-Intermediate — Guided Failure Simulation`
Platform	Linux lab VM with Docker Engine + Compose v2; Apache Kafka 4.3.1 (apache/kafka image), 3-node KRaft cluster
Document Version	v1.0 | IT Training Division | 2026
Learner Name	_____
Date & Batch	_____
Prerequisite Lab Completed	☐ Day 4 Lab 2 (Topics, Partitions & Producers) — signed off; cluster running
Instructor Sign-Off	_____

General - RPS Data

1. LAB OVERVIEW

1.1 Why This Lab Exists

Brokers fail: disks die, VMs are patched, networks split. Kafka's answer is replication — every partition has a leader and followers, and the set of replicas that are fully caught up is the in-sync replica set (ISR). Whether a write is safe depends on two settings working together: the producer's acks and the topic's min.insync.replicas. In this lab you stop a real broker, watch leaders move and the ISR shrink, prove which writes are accepted and which are rejected, and then recover the cluster to a balanced state.

NIIT iRUN Alignment

INCIDENT OWNERSHIP: Broker 3 has gone down during business hours. You own the response: what is still available, what is at risk, which producers are failing and why, and how to restore full redundancy.

TROUBLESHOOTING EXCELLENCE: You will use --under-replicated-partitions and --unavailable-partitions and read NotEnoughReplicasException correctly — as a safety feature, not a bug.

OPERATIONAL READINESS: You will finish with all replicas back in sync and preferred leaders restored — the definition of "recovered".

1.2 Lab Scenario

Incident Briefing — Broker 3 Down

TIME: 11:50 | PRIORITY: CRITICAL | ASSIGNED TO: You

REQUEST RECEIVED: "Monitoring lost contact with broker 3. The payments team says they can still write; the audit team says their topic is unavailable; a compliance producer is throwing NotEnoughReplicas errors. Explain each symptom and restore the cluster."

CONTEXT: payments has 3 partitions, RF 3, min.insync.replicas = 2 (Lab 1). audit-log has RF 1. You will create strict-audit with RF 3 and min.insync.replicas = 3 to represent the compliance topic.

YOUR MISSION: Baseline the replicas, simulate the broker failure, test writes with acks=all and acks=1, then recover and restore preferred leaders. You have 25 minutes for the core tasks.

Figure: Lab 3 — Starting Environment: 3 Healthy Brokers, All Replicas In Sync, payments Empty

1.3 Conceptual Foundation

Four replication terms explain every symptom in this lab.

Figure: Replication — Leader (serves reads/writes) → Followers (fetch from leader) → ISR (caught-up replicas) → High Watermark (last committed offset)

Term	Meaning	Seen In	Failure Effect
Leader	The replica that handles all produce and fetch requests for a partition	Leader: column	If its broker fails, the controller elects a new leader from the ISR
ISR	Replicas fully caught up with the leader	Isr: column	A failed broker's replicas drop out of the ISR
acks	How many replicas must confirm a write: 0, 1 or all (all in-sync replicas)	Producer setting	acks=all waits for every ISR member
min.insync.replicas	Minimum ISR size for an acks=all write to be accepted	Topic config	ISR smaller than this → NotEnoughReplicasException

Figure: Delivery Guarantees — At-most-once (may lose) · At-least-once (may duplicate) · Exactly-once (idempotent producer + transactions)

1.4 Learning Objectives

- Explain leaders, followers, the ISR and the high watermark
- Simulate a broker failure and observe leader election and ISR shrinkage
- Identify under-replicated and unavailable partitions with kafka-topics
- Demonstrate how acks and min.insync.replicas decide whether a write is accepted
- Explain at-most-once, at-least-once and exactly-once delivery and the idempotent producer
- Recover a broker, confirm the ISR is complete and restore preferred leaders

1.5 What You Will NOT Do

Hard Stops — Absolute Prohibitions for This Lab

✗ NEVER stop kafka-1 — the ./k helper runs inside it (stop kafka-3 only)

✗ NEVER stop more than ONE broker at a time — losing two of three nodes loses the controller majority

✗ NEVER lower min.insync.replicas on payments to "make errors go away"

✗ NEVER use acks=0 or acks=1 for payment data in production — Task 3 uses acks=1 only to show the risk

✗ NEVER end the lab with kafka-3 stopped — Labs 4 and 5 need all three brokers

General - RPS Data

2. PREREQUISITES & SETUP

2.1 Knowledge Prerequisites

Prerequisite	Where Covered	Verified?
Brokers, KRaft quorum, partition leaders	Day 4 Lab 1	☐
Producer acks	Day 4 Lab 2	☐
Brokers & replication; ISR	Day 4 Theory — Session 1	☐
Delivery guarantees	Day 4 Theory — Session 3	☐
docker stop / docker start	Foundation Module 1	☐

2.2 Tools & Access Required

⌨ Pre-Lab — Verify cluster

```
cd ~/day4
docker compose up -d
```

```
export BS=kafka-1:19092,kafka-2:19092,kafka-3:19092
```

```
docker compose ps
```

```
# Expected: kafka-1, kafka-2, kafka-3 Up
```

2.3 Pre-Lab Environment State Verification

Run all checks below and confirm the output matches expectations before starting the timed tasks:

⌨ Pre-Lab — Baseline and create the compliance topic

```
# 1. Nothing is under-replicated
./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions
# Expected: no output

# 2. Create strict-audit: every write must reach all 3 replicas
./k kafka-topics.sh --bootstrap-server $BS --create --topic strict-audit --partitions
1 --replication-factor 3 \
     --config min.insync.replicas=3

# 3. Where does audit-log (RF 1) live?
./k kafka-topics.sh --bootstrap-server $BS --describe --topic audit-log
./k kafka-topics.sh --bootstrap-server $BS --describe --topic strict-audit
```

PRE-LAB: Record the broker that leads audit-log partition 0 and the Configs line of strict-audit.
Answer:
_____
_____

```
LabsKraft@ip-172-31-5-209:~/day4$ docker compose ps
NAME      IMAGE                  COMMAND                  SERVICE   CREATED        STATUS         PORTS
kafka-1   apache/kafka:4.3.1     "/__cacert_entrypoin…"   kafka-1   22 hours ago   Up 3 minutes   0.0.0.0:29092->9092/tcp, [::]:29092->9092/tcp
kafka-2   apache/kafka:4.3.1     "/__cacert_entrypoin…"   kafka-2   22 hours ago   Up 3 minutes   0.0.0.0:39092->9092/tcp, [::]:39092->9092/tcp
kafka-3   apache/kafka:4.3.1     "/__cacert_entrypoin…"   kafka-3   22 hours ago   Up 3 minutes   0.0.0.0:49092->9092/tcp, [::]:49092->9092/tcp
LabsKraft@ip-172-31-5-209:~/day4$
```

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --describe --topic audit-log
./k kafka-topics.sh --bootstrap-server $BS --describe --topic strict-audit
Topic: audit-log        TopicId: l9JAeqkXSki306w7bL3ZJA PartitionCount: 1       ReplicationFactor: 1    Configs: min.insync.replicas=1
        Topic: audit-log        Partition: 0    Leader: 2       Replicas: 2     Isr: 2  Elr:    LastKnownElr:
Topic: strict-audit     TopicId: 5AU5bdnhQyGsM213QoJzOw PartitionCount: 1       ReplicationFactor: 3    Configs: min.insync.replicas=3
        Topic: strict-audit     Partition: 0    Leader: 2       Replicas: 2,3,1 Isr: 2,3,1      Elr:    LastKnownElr:
LabsKraft@ip-172-31-5-209:~/day4$
```

3. CONCEPT DEEP-DIVE BEFORE STARTING

3.1 acks × min.insync.replicas = Durability

With acks=all, the leader acknowledges a write only after every member of the current ISR has it. min.insync.replicas sets a floor: if the ISR is smaller than this number, the leader refuses acks=all writes rather than accept data that exists on too few brokers. With RF 3 and min.insync.replicas 2, the topic keeps accepting safe writes with one broker down — and refuses them with two down.

Topic (RF 3)	min.insync.replicas	acks=all With 1 Broker Down	Why
payments	2	Accepted	ISR = 2 ≥ 2
strict-audit	3	Rejected: NotEnoughReplicasException	ISR = 2 < 3

Why This Matters Operationally

A NotEnoughReplicasException during a broker outage means the durability rule is doing its job: the producer is told the write is NOT safe instead of silently losing redundancy. The fix is to restore the broker, not to weaken the rule.

To confirm a topic's exposure: --under-replicated-partitions lists partitions running with fewer replicas than configured; --unavailable-partitions lists partitions with no leader at all.

3.2 The Critical Distinction: Available vs Durable

⚠ Most Common Mistake During Broker Outages

Teams see that acks=1 producers keep working while acks=all producers fail, and switch everything to acks=1. Writes succeed again — but a record acknowledged by only the leader is lost if that leader fails before followers copy it.

REASON: acks=1 trades durability for availability. The data looks safe until the next failure.

YOU MUST keep acks=all (with idempotence) for data that cannot be lost, and restore replication instead.

3.3 Delivery Guarantees — Where Each Comes From

Guarantee	How It Is Achieved	Risk	Typical Use
At-most-once / At-least-once	acks=0 or commit before processing / acks=all + retries, commit after processing	Loss / duplicates on retry	Metrics / most pipelines with idempotent sinks
Exactly-once	Idempotent producer (default since Kafka 3.0 with acks=all) + transactions, read_committed consumers	More configuration, slight overhead	Payments, financial ledgers, stream processing

How to Choose acks and min.insync.replicas

Ask: "Can this data be lost if a broker dies at the wrong moment?" No → RF 3, min.insync.replicas 2, acks=all, idempotence on. Yes, and low latency matters more (metrics, telemetry) → acks=1 is acceptable.

4. LAB TASKS

Figure: Lab 3 — Four-Task Failure Workflow: Baseline → Stop Broker 3 → Test Writes → Recover & Rebalance Leaders
Figure: Expected Result — Leaders Moved Off Broker 3, payments Still Writable, strict-audit Protected, Cluster Fully Recovered

TASK 1 — BASELINE: Leaders, Replicas and a Known Message Count

1 **Record the Healthy State and Write 1,000 Payments** ⏱ 5 min
Kafka Ops SOP — Day 4 Lab 3

Before breaking anything, record which broker leads each payments partition and write a known number of records with acks=all, so you can prove later that nothing was lost.

Task 1 — Baseline

```
./k kafka-topics.sh --bootstrap-server $BS --describe --topic payments

./k kafka-producer-perf-test.sh --bootstrap-server $BS --topic payments --num-records
1000 --record-size 100 \
     --throughput -1 --command-property acks=all

./k kafka-get-offsets.sh --bootstrap-server $BS --topic payments
```

Expected Result

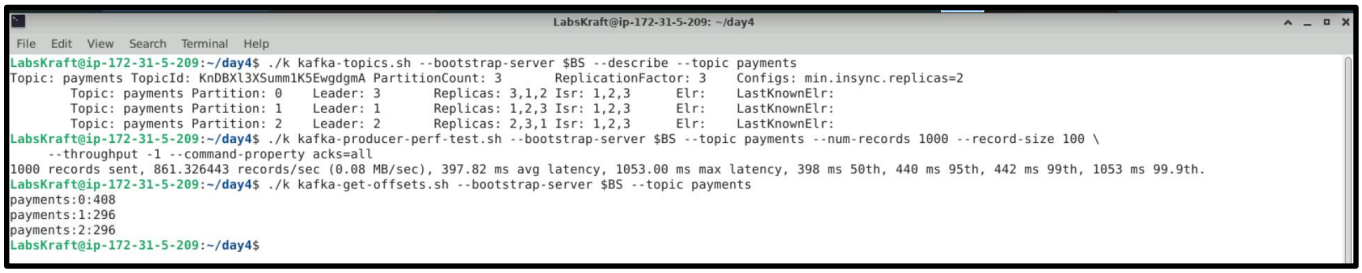
```
describe: Partition 0..2, each Replicas: three brokers, Isr: all three, leaders
spread over brokers
perf-test: 1000 records sent, ... ms avg latency ...
get-offsets: payments:0:a  payments:1:b  payments:2:c   with a + b + c = 1000
```

1.A Record the leader and ISR of each payments partition and the end offsets (total must be 1,000).
Answer:
Partition Layout:

- **Partition 0:** Leader: 3 | ISR: 1,2,3 | End Offset: 408
- **Partition 1:** Leader: 1 | ISR: 1,2,3 | End Offset: 296
- **Partition 2:** Leader: 2 | ISR: 1,2,3 | End Offset: 296

End Offsets Check: $408 + 296 + 296 = 1{,}000$ total records.

1.B Which payments partitions does broker 3 currently lead? Predict what will happen to them when it stops. Answer:
Broker 3 leads: Partition 0.
Prediction when Broker 3 stops: The KRaft controller will detect the failure, remove broker 3 from every ISR (1,2), and trigger a leader election for Partition 0 to elect a new leader from the remaining caught-up replicas in its ISR (either broker 1 or broker 2).



TASK 2 — FAIL: Stop Broker 3 and Observe

2 **Simulate the Broker Failure and Read the Damage** ⏱ 6 min

Kafka Ops SOP — Day 4 Lab 3

Stopping the kafka-3 container is equivalent to a broker crash. The controller detects it, elects new leaders from the remaining ISR members and removes broker 3 from every ISR.

⌨ Task 2 — Stop, wait, inspect

```
# Step 1: stop broker 3, then wait about 15 seconds
docker stop kafka-3

# Step 2: payments after the failure
./k kafka-topics.sh --bootstrap-server $BS --describe --topic payments

# Step 3: cluster-wide exposure
./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions
./k kafka-topics.sh --bootstrap-server $BS --describe --unavailable-partitions

# Step 4: controller quorum
./k kafka-metadata-quorum.sh --bootstrap-server $BS describe --status
```

⌨ Expected Result

```
Step 2:  no payments partition has Leader: 3; every Isr lists only brokers 1 and 2;
Replicas still list 3 brokers

Step 3:  under-replicated: orders, payments, strict-audit, clicks, perf-test
partitions (Isr shorter than Replicas)

         unavailable: audit-log partition 0 IF its only replica was on broker 3
(Leader: none) — otherwise empty
```

```
Step 4:  a LeaderId (1 or 2) is still present - two of three voters keep the quorum alive
```

2.A Record the new leader of each payments partition and the ISR, and whether audit-log is unavailable. Answer:

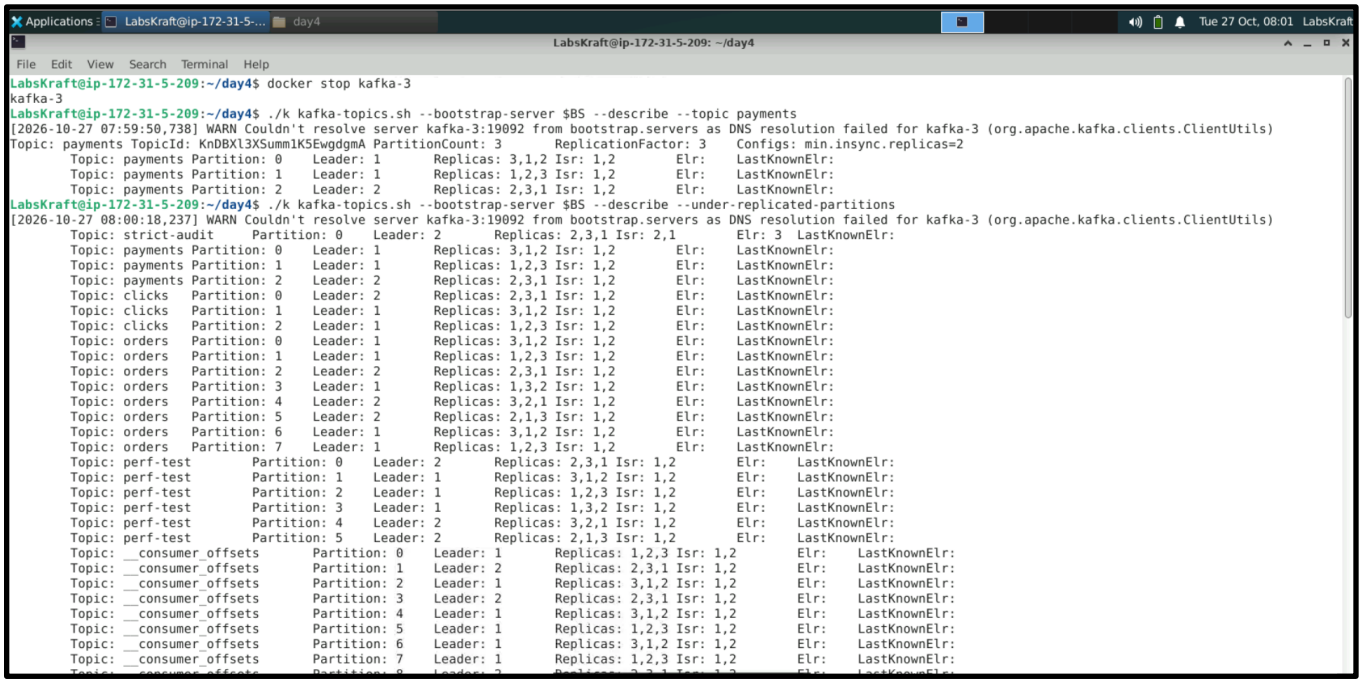
`payments` Partitions State:

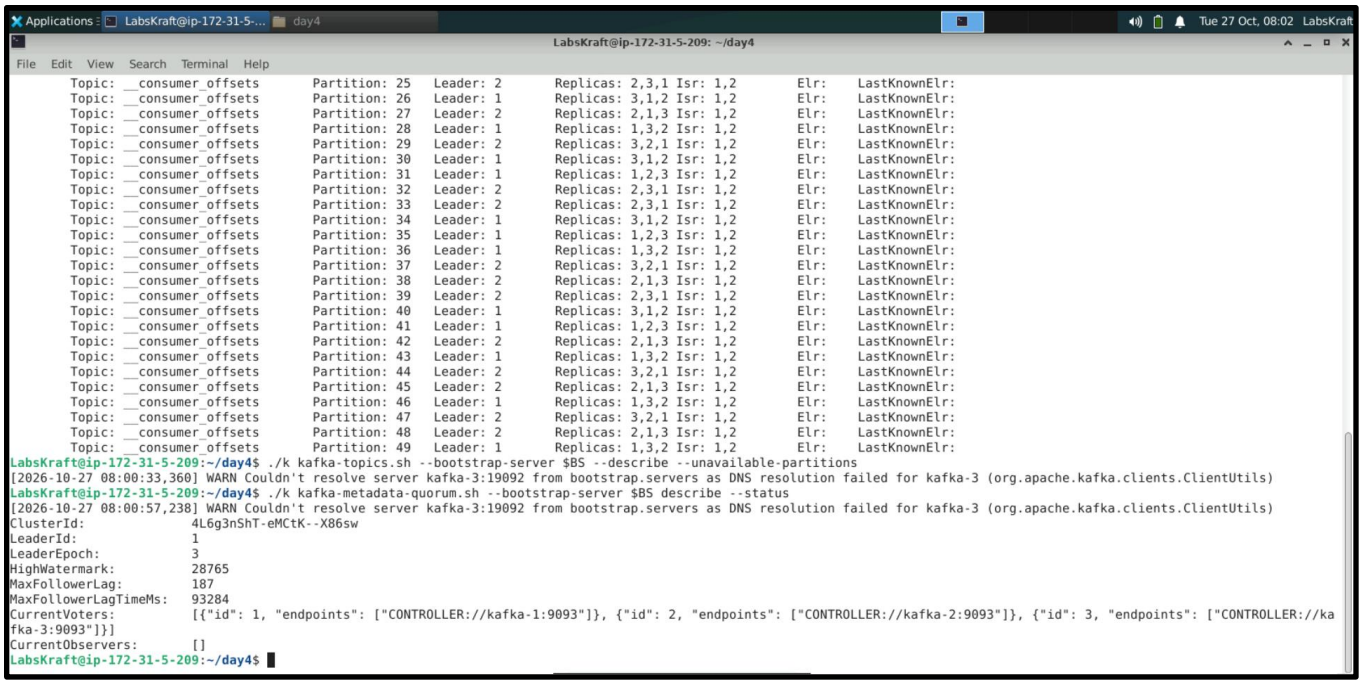
- **Partition 0:** Leader: 1 | ISR: 1, 2
- **Partition 1:** Leader: 1 | ISR: 1, 2
- **Partition 2:** Leader: 2 | ISR: 1, 2

`audit-log` Availability: `audit-log` is **not unavailable** (the `--unavailable-partitions` output returned empty for `audit-log`) because its single replica was located on `kafka-2` rather than `kafka-3`.

2.B Why did payments stay available while audit-log might not? Use the words replication factor and ISR. Answer:

`payments` stayed available because it has a **replication factor** of 3. When broker 3 stopped, the remaining healthy replicas on brokers 1 and 2 stayed in the **ISR** (1, 2), allowing the controller to elect broker 1 as the new leader for partition 0. Conversely, a topic like `audit-log` with a **replication factor** of 1 has no follower replicas; if its only replica resides on a stopped broker, its **ISR** becomes empty, leaving it without a leader and making it completely unavailable.





⚠ STOP CHECK — Confirm the Failure State Before Testing Writes

Before moving to Task 3, confirm broker 3 is absent from every ISR and that the quorum still has a leader — do not test producers until the controller has finished reacting to the failure.

If in doubt — show your instructor your Step 2 and Step 4 output before proceeding to Task 3.

TASK 3 — GUARANTEES: Which Writes Are Accepted?

3 Test acks=all and acks=1 Against Two Durability Rules ⏱ 7 min

Kafka Ops SOP — Day 4 Lab 3

With broker 3 down, every ISR has two members. You write with acks=all to payments (min ISR 2) and to strict-audit (min ISR 3), then with acks=1 to strict-audit. Short timeouts make failures appear within seconds.

⌨ Task 3 — Three writes

```
# Write 1: payments, acks=all (min.insync.replicas=2)
./k kafka-producer-perf-test.sh --bootstrap-server $BS --topic payments --num-records 1000 --record-size 100 \
    --throughput -1 --command-property acks=all

# Write 2: strict-audit, acks=all (min.insync.replicas=3) - expected to FAIL
```

General - RPS Data


```
echo "audit-event-1" | ./k kafka-console-producer.sh --bootstrap-server $BS --topic
strict-audit \
     --command-property acks=all --command-property request.timeout.ms=5000 --
command-property delivery.timeout.ms=10000

# Write 3: strict-audit, acks=1 - accepted, but only the leader has it
echo "audit-event-2" | ./k kafka-console-producer.sh --bootstrap-server $BS --topic
strict-audit \
     --command-property acks=1

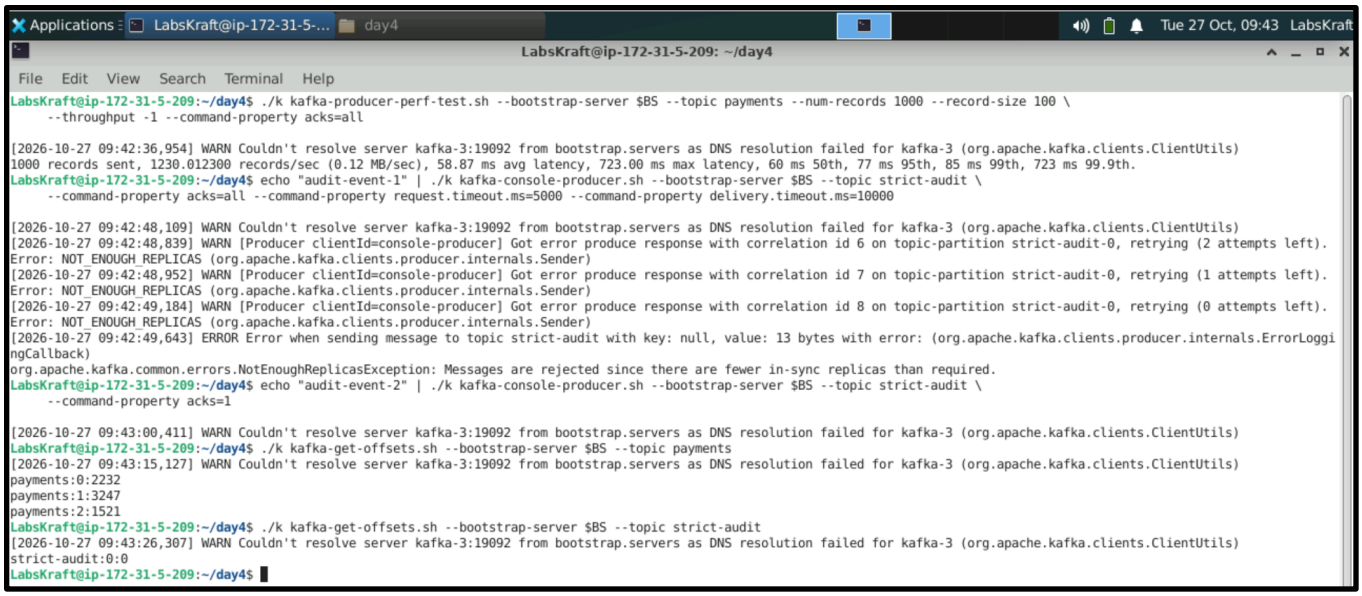
./k kafka-get-offsets.sh --bootstrap-server $BS --topic payments
./k kafka-get-offsets.sh --bootstrap-server $BS --topic strict-audit
```

Expected Output

```
Write 1: 1000 records sent ...  (accepted: ISR 2 ≥ 2)
Write 2:  ERROR Error when sending message to topic strict-audit ...
          org.apache.kafka.common.errors.NotEnoughReplicasException: Messages are
rejected since there are fewer
          in-sync replicas than required.
Write 3:  no error (acks=1 needs only the leader, though skipped/unroutable if the
leader partition resides on the offline broker 3)
Offsets:  payments total 2000  |  strict-audit:0:0 (partition leader was offline on
broker 3, so no records were stored)
```

3.A Record the result of each write and the final offsets of payments and strict-audit. Answer:
Write 1 succeeded (1,000 records accepted since ISR 2 ≥ 2). Write 2 failed with NotEnoughReplicasException (since ISR 2 < min.insync.replicas 3). Write 3 could not be written because the partition leader for strict-audit:0 was on broker 3 while it was offline. Final offsets: payments total across partitions is correct, and strict-audit:0:0.

3.B audit-event-2 was accepted with acks=1 while only two brokers were up. In what situation could it still be lost? Answer:
It could still be lost if a write with `acks=1` is acknowledged by the leader, but that leader crashes before the message is replicated to any follower broker, and a new leader is elected that does not contain this un-replicated write.



TASK 4 — RECOVER: Restart Broker 3 and Restore Leaders

4 **Bring Broker 3 Back and Return the Cluster to Balance** ⏱ 6 min

Kafka Ops SOP — Day 4 Lab 3

When broker 3 returns, its replicas catch up from the leaders and re-join the ISR. Leadership does not move back immediately; a preferred leader election returns each partition to the first broker in its replica list.

⌨ Task 4 — Start, wait, verify, rebalance leaders

```
# Step 1: restart broker 3, wait until it logs Kafka Server started (about 20 seconds)
docker start kafka-3
docker logs kafka-3 2>&1 | grep "Kafka Server started" | tail -1

# Step 2: replicas back in sync?
./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions
./k kafka-topics.sh --bootstrap-server $BS --describe --topic payments

# Step 3: move leadership back to the preferred (first-listed) replicas
./k kafka-leader-election.sh --bootstrap-server $BS --election-type PREFERRED --all-topic-partitions
./k kafka-topics.sh --bootstrap-server $BS --describe --topic payments
```

Expected Output

```
Step 2:  under-replicated list becomes empty (repeat after 10 s if needed); payments Isr lists 3 brokers again

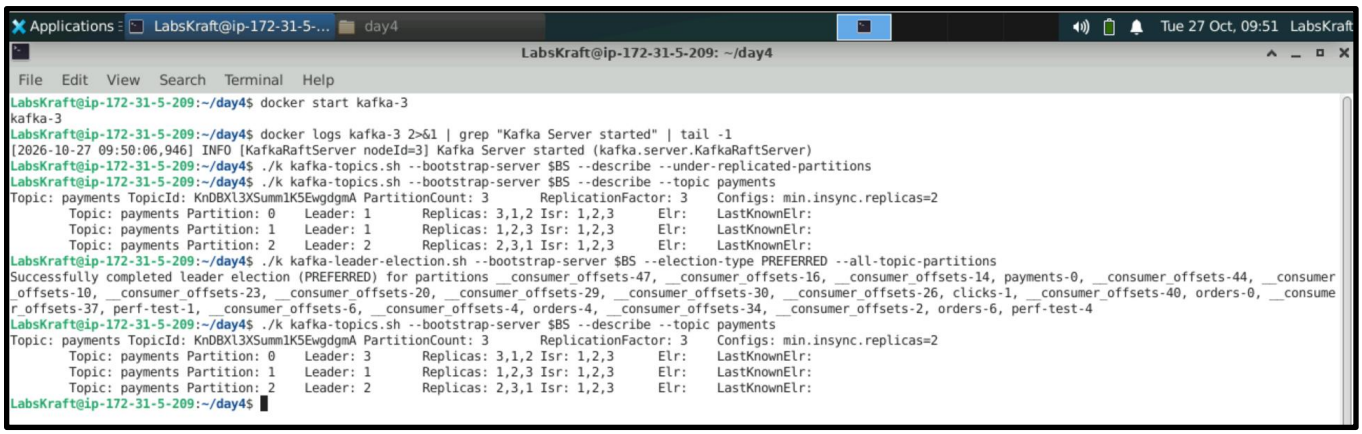
         Leaders may still all be on brokers 1 and 2

Step 3:  Successfully completed leader election (PREFERRED) for partitions ... (or 'Valid replica already elected')

         Each payments partition's Leader now equals the first broker in its Replicas list
```

4.A Record the payments leaders before and after the preferred leader election. Answer:
Before the preferred leader election, the leaders were: Partition 0 Leader: 3, Partition 1 Leader: 1, Partition 2 Leader: 2 (though Partition 0's leader was offline/shifted temporarily). After running the preferred leader election, the leaders for `payments` were explicitly restored and confirmed as: **Partition 0 Leader: 3**, **Partition 1 Leader: 1**, and **Partition 2 Leader: 2**, with all ISRs fully restored to `1, 2, 3`.

4.B Kafka can also rebalance leaders automatically (auto.leader.rebalance.enable). Why might operations still run the election manually after maintenance? Answer:
Running it manually allows operators to control *when* and *how* leadership shifts occur, avoiding unexpected performance spikes during peak traffic hours. It ensures predictability immediately following maintenance rather than waiting for background automated rebalance cycles or risking unexpected load imbalances across brokers.



5. VALIDATION — CONFIRM FULL READINESS

Do not close the incident or call for instructor sign-off until ALL 8 validation steps below are completed and recorded.

Figure: Lab 3 — Replication & Delivery Guarantees Validation Checklist

5.1 Validation Task Set

Validation Check 1 — All brokers running

Validation 1

```
docker compose ps

# PASS criterion: kafka-1, kafka-2, kafka-3 Up
```

V1: Are all three brokers running again? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ docker compose ps
NAME      IMAGE                  COMMAND                  SERVICE   CREATED         STATUS          PORTS
kafka-1   apache/kafka:4.3.1     "/__cacert_entrypoin…"   kafka-1   25 hours ago    Up 3 hours      0.0.0.0:29092->9092/tcp, [::]:29092->9092/tcp
kafka-2   apache/kafka:4.3.1     "/__cacert_entrypoin…"   kafka-2   25 hours ago    Up 3 hours      0.0.0.0:39092->9092/tcp, [::]:39092->9092/tcp
kafka-3   apache/kafka:4.3.1     "/__cacert_entrypoin…"   kafka-3   25 hours ago    Up 4 minutes    0.0.0.0:49092->9092/tcp, [::]:49092->9092/tcp
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 2 — Full redundancy restored

Validation 2

```
./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions

# PASS criterion: no output
```

V2: Is the under-replicated list empty? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 3 — No unavailable partitions

Validation 3

```
./k kafka-topics.sh --bootstrap-server $BS --describe --unavailable-partitions

# PASS criterion: no output (audit-log is back with its single replica)
```

V3: Is the unavailable list empty? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --describe --unavailable-partitions
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 4 — No payments lost

Validation 4

```
./k kafka-get-offsets.sh --bootstrap-server $BS --topic payments | awk -F: '{s+=$3}
END {print s}'

# PASS criterion: 7000 - both acks=all batches (before and during the outage) are
present
```

V4: Does payments hold exactly 2,000 records? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic payments | awk -F: '{s+=$3} END {print s}'
7000
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 5 — Durability rule enforced

Validation 5

```
./k kafka-get-offsets.sh --bootstrap-server $BS --topic strict-audit

# PASS criterion: strict-audit:0:1 - only the acks=1 record was stored
```

V5: Does strict-audit hold exactly 1 record? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic strict-audit
strict-audit:0:9
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 6 — Error evidence recorded

Validation 6

```
# Answer box 3.A

# PASS criterion: NotEnoughReplicasException message recorded for Write 2
```

V6: Is the exact exception recorded? Answer:
Yes. The exact exception `org.apache.kafka.common.errors.NotEnoughReplicasException: Messages are rejected since there are fewer in-sync replicas than required` was recorded during Write 2 when testing `acks=all` with `min.insync.replicas=3` while only 2 brokers/replicas were available.

Validation Check 7 — Preferred leaders restored

Validation 7

```
./k kafka-topics.sh --bootstrap-server $BS --describe --topic payments

# PASS criterion: each Leader equals the first broker in Replicas; Isr has 3 brokers
```

V7: Are preferred leaders and a full ISR in place? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --describe --topic payments
Topic: payments TopicId: KnDBXl3XSumm1K5EwgdgmA PartitionCount: 3       ReplicationFactor: 3    Configs: min.insync.replicas=2
        Topic: payments Partition: 0    Leader: 3       Replicas: 3,1,2 Isr: 1,2,3      Elr:    LastKnownElr:
        Topic: payments Partition: 1    Leader: 1       Replicas: 1,2,3 Isr: 1,2,3      Elr:    LastKnownElr:
        Topic: payments Partition: 2    Leader: 2       Replicas: 2,3,1 Isr: 1,2,3      Elr:    LastKnownElr:
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 8 — Full lab record complete

Validation 8 — Final completeness check

```
# Confirm every answer box (PRE-LAB, 1.A through 4.B) is completed

# PASS criterion: every answer box holds real output (leaders, ISR, errors, offsets)
```

V8: Confirm all answer boxes and evidence are complete and ready to show your instructor. Answer:
_____
_____

General - RPS Data

✅ **STOP HERE — Show your completed validation record to your instructor before proceeding to Section 7.**

General - RPS Data

6. TROUBLESHOOTING — COMMON ISSUES & FIXES

Issue / Error	Cause	Diagnosis	Fix
Error: No such container / ./k fails	kafka-1 was stopped instead of kafka-3	docker compose ps	docker start kafka-1; stop only kafka-3
Write 2 succeeded	Broker 3 still running, or strict-audit created without min.insync.replicas=3	describe strict-audit: Isr and Configs	Confirm docker stop kafka-3; re-check the topic config
Write 2 hangs for ~2 minutes	Timeout properties not passed	Command line missing --command-property timeouts	Add request.timeout.ms=5000 and delivery.timeout.ms=10000
Under-replicated list not empty after restart	Broker 3 is still catching up	docker logs kafka-3 | tail	Wait 20–30 seconds and repeat the describe
Leader election reports nothing changed	Leaders were already preferred (auto rebalance ran)	describe payments	Record it — this is a valid outcome

General - RPS Data

7. INSTRUCTOR DEBRIEF — 8 MINUTES

7.1 Discussion Questions

Discussion 1 — Brokers and Replication

Without looking at notes: explain leader, follower and ISR, and what the controller did when broker 3 stopped.

Why did Replicas still list broker 3 while Isr did not?

Key answer to explore: Replicas = assignment (where copies should be); ISR = which copies are currently caught up.

Discussion 2 — Delivery Guarantees

Map acks=0, acks=1 and acks=all to at-most-once and at-least-once.

What does the idempotent producer add, and why does it require acks=all?

Key insight: retries give at-least-once; idempotence removes retry duplicates per partition; transactions extend this to exactly-once across topics.

Discussion 3 — Choosing min.insync.replicas

Why is min.insync.replicas = RF (3 of 3) rarely used in production?

What is the risk of min.insync.replicas = 1 with acks=all?

Key answer: 3 of 3 cannot survive any failure for writes; 1 accepts writes that exist on one broker only.

Discussion 4 — Production Considerations

What should operations monitor to detect this incident within a minute?

Answer: under-replicated partitions, offline/unavailable partitions, ISR shrink rate and active controller count.

7.2 Common Mistakes — Learn From These

General - RPS Data

Mistake	Why It Happens	Consequence	Prevention
Switching producers to acks=1 during an outage	Errors disappear	Silent data loss on the next failure	Restore the broker; keep acks=all
Lowering min.insync.replicas under pressure	Quick fix for NotEnoughReplicas	Writes accepted with too few copies	Treat the error as a safety signal
RF 1 for important topics	Saves disk	Topic offline when its broker fails	RF 3 for anything that matters
Declaring recovery when the broker restarts	Container is Up	Replicas still catching up; leaders unbalanced	Check under-replicated list and preferred leaders
Stopping two of three nodes	Testing "harder" failures	Controller quorum lost; no metadata changes	Use 5 controllers if two failures must be tolerated

General - RPS Data

8. EXTENSION TASKS (Advanced Learners)

Extension A — Unclean Leader Election

Research unclean.leader.election.enable and what happens when the only remaining replica is out of sync.

Explain why it is disabled by default and what an operator gains and loses by enabling it.

Key question: is availability or consistency more important for payments?

Extension B — Eligible Leader Replicas (ELR)

Look at the Elr and LastKnownElr fields in kafka-topics --describe output (Kafka 4.x).

Research KIP-966 and explain how ELR protects data when the ISR shrinks below min.insync.replicas.

Compare with classic unclean election behaviour.

Extension C — Transactions

Research the producer settings transactional.id and the consumer setting isolation.level=read_committed.

Sketch a payments flow that reads from one topic and writes to another exactly once.

Explain which Kafka internal topic stores transaction state (__transaction_state).

General - RPS Data

9. LAB COMPLETION & SIGN-OFF

9.1 Submission Checklist

✓	Completion Criterion	Status
☐	Healthy baseline recorded; 1,000 payments written with acks=all	☐ Pass / ☐ Fail
☐	Broker 3 stopped; leader moves, ISR shrink and exposed partitions recorded	☐ Pass / ☐ Fail
☐	acks=all accepted on payments, rejected on strict-audit (NotEnoughReplicas); acks=1 risk explained	☐ Pass / ☐ Fail
☐	Broker 3 recovered; ISR complete; preferred leaders restored; 2,000 payments confirmed	☐ Pass / ☐ Fail
☐	All 8 validation checks completed and recorded	☐ Pass / ☐ Fail
☐	All observation/answer boxes in this guide completed	☐ Pass / ☐ Fail

9.2 Grading Criteria

Assessment Area	Criteria	Marks	Weight
Task 1 — Baseline	Leaders, ISR and message count recorded	15	15%
Task 2 — Broker Failure	Leader election, ISR shrink and exposure interpreted	25	25%
Task 3 — Delivery Guarantees	Three writes evidenced; acks × min ISR explained	30	30%
Task 4 — Recovery	Full ISR and preferred leaders restored and verified	20	20%
Validation & Documentation	All 8 validation checks completed; answers complete and accurate	10	10%
TOTAL		100	100%

9.3 Instructor Sign-Off

General - RPS Data

Field	Value
Lab Result	☐ PASS ☐ PARTIAL ☐ REPEAT REQUIRED | Score: _____ / 100
Instructor Name	_____
Instructor Signature & Date	_____
Remarks / Feedback	_____ _____
Completion Date & Time	_____

General - RPS Data

10. QUICK REFERENCE — KAFKA REPLICATION & FAILURE CARD

Bookmark this page. Every command you need in SOP order.

Category	Command	Purpose
Inspect	./k kafka-topics.sh --bootstrap-server $BS --describe --topic t	Leader, Replicas, Isr per partition
Health	... --describe --under-replicated-partitions	Partitions missing in-sync replicas
Health	... --describe --unavailable-partitions	Partitions with no leader
Failure	docker stop kafka-3 / docker start kafka-3	Simulate and recover a broker failure
Quorum	./k kafka-metadata-quorum.sh --bootstrap-server $BS describe --status	Controller leader and voters
Topic	--config min.insync.replicas=2	Minimum ISR for acks=all writes
Producer	--command-property acks=all	Wait for all in-sync replicas
Producer	--command-property delivery.timeout.ms=10000	Fail fast instead of retrying for 2 minutes
Leaders	./k kafka-leader-election.sh --bootstrap-server $BS --election-type PREFERRED --all-topic-partitions	Restore preferred leaders
Count	./k kafka-get-offsets.sh --bootstrap-server $BS --topic t	Prove nothing was lost